Ecole Supérieure
d'Informatique et du Numérique
COLLEGE OF ENGINEERING & ARCHITECTURE

COMPTE RENDU

SDN & Network Softwarization — TP5 — Open vSwitch sans contrôleur : flux manuels et captures Wireshark
4e année Cybersécurité — École Supérieure d'Informatique et du Numérique (ESIN)
Collège d'Ingénierie & d'Architecture (CIA)

Étudiant : HATHOUTI Mohammed Taha

Filière : Cybersécurité

Année : 2026/2027

Enseignant : M. BOUHORMA

Date : 28 octobre 2026

Table des matières

Cadre du TP **2**

1 Comprendre le laboratoire **3**

2 Démarrage de l'environnement **3**
2.1 Terminal 1 : création de la topologie 3
2.2 Terminal 2 : paramètres OVS 5
2.3 Wireshark 6

3 Expérience 1 — Switch en mode `secure` avec table vide **6**

4 Expérience 2 — Retrouver le comportement d'un switch classique **8**

5 Expérience 3 — ARP puis forwarding IPv4 manuel **10**
5.1 Autoriser seulement ARP 10
5.2 Autoriser IPv4 selon l'adresse de destination 11

6 Expérience 4 — ACL ICMP et priorités **14**
6.1 Bloquer tout ICMP de h1 vers h3 14
6.2 Supprimer précisément cette ACL 15
6.3 Bloquer uniquement l'Echo Request (ICMP type 8) 15

7 Expérience 5 — ACL TCP sur le service HTTP 8080 **17**
7.1 Démarrer le serveur et vérifier le trafic autorisé 17
7.2 Bloquer TCP vers 10.0.0.3 :8080 17
7.3 Supprimer l'ACL et restaurer HTTP 18

8 Synthèse Wireshark — comparer les états A à D **19**
8.1 Observation de `pingall` et des tables 0 et 1 20

9 Clôture du TP **21**

10 Questions de validation **22**

11 Synthèse **22**

Cadre du TP

Plateforme	**Outils**	**Mode**
Ubuntu (portable `Taha-inspiron-16`)	Mininet, Open vSwitch (`ovs-vsctl`, `ovs-ofctl`), OpenFlow 1.3, Wireshark graphique	Individuel
Approche	**Switch / contrôleur**	**Livrable**
Flux installés à la main avec `ovs-ofctl`, cinq expériences progressives observées dans Wireshark	`OVSSwitch` (`ovsk`) en OpenFlow 1.3, `fail-mode secure`, aucun contrôleur	Compte rendu avec captures

TABLE 1 – Cadre général du TP

Question de départ : *un lien fonctionne, les adresses IP sont correctes, mais un ping échoue. Est-ce forcément un problème physique ?*

Objectifs :

- distinguer Open vSwitch (commutateur logiciel) et OpenFlow (règles de traitement) ;
- créer trois hôtes et un switch OVS sans contrôleur, en OpenFlow 1.3 et `fail-mode secure` ;
- afficher, ajouter et supprimer des flux avec `ovs-ofctl` depuis un second terminal ;
- distinguer ARP, ICMP et TCP et relier chaque paquet à une règle et à une priorité ;
- observer dans Wireshark, à chaque expérience, la présence ou l'absence du paquet à l'entrée (`s1-eth1`) et à la sortie (`s1-eth3`) du switch ;
- comparer la commutation `NORMAL`, les flux IPv4 explicites, les ACL ICMP et TCP, puis vérifier le retour du trafic après suppression des règles.

Méthode constante :

1. Terminal 2 : installer ou supprimer la règle ;
2. Wireshark : *Capture* → *Redémarrer*, puis appliquer le filtre d'affichage ;
3. Terminal 1 (`mininet>`) : générer le trafic ;
4. Wireshark : comparer l'interface d'entrée et l'interface de sortie ;
5. Terminal 2 : lire `n_packets` avec `dump-flows`.

Note sur l'environnement. Manipulations faites sur mon portable Ubuntu, dans le dossier `~/Bureau/UIR/4A/S7/SDN/TP5`, avec trois fenêtres : le Terminal 1 pour Mininet, le Terminal 2 pour `ovs-vsctl` et `ovs-ofctl`, et Wireshark en mode graphique, lancé sans `sudo`, capturant simultanément `s1-eth1` et `s1-eth3` avec un filtre de capture vide. Toutes les commandes `ovs-ofctl` portent l'option `-O OpenFlow13`.

1 Comprendre le laboratoire

Dans l'architecture SDN, les applications et le contrôleur appartiennent au plan de contrôle, tandis que les commutateurs traitent les paquets dans le plan de données. Dans ce TP il n'y a **pas de contrôleur** : la topologie `single,3` relie `h1`, `h2` et `h3` à `s1`, et les règles sont installées manuellement depuis le Terminal 2.

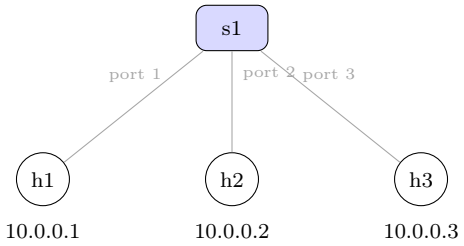


FIGURE 1 – Topologie du TP : `single,3`

Hôte	Adresse IP	Interface OVS	Port OpenFlow
`h1`	10.0.0.1	`s1-eth1`	1
`h2`	10.0.0.2	`s1-eth2`	2
`h3`	10.0.0.3	`s1-eth3`	3

TABLE 2 – Plan d'adressage et ports attendus

Principe de sélection. Le paquet entre dans `s1` ; parmi les règles qui correspondent, celle de **priorité la plus élevée** est choisie et son action est exécutée. Sans règle applicable, le paquet n'est pas transféré.

Fenêtre	Contenu	Commandes
Terminal 1	Mininet (`mininet>`), maintenu ouvert pendant tout le TP	`sudo mn -c`, `sudo mn ...`, `net`, `pingall`, `h1 ping ...`, `h3 python3 ...`
Terminal 2	Terminal Ubuntu indépendant	`sudo ovs-vsctl ...`, `sudo ovs-ofctl -O OpenFlow13 ...`
Wireshark	Application graphique, captures de `s1-eth1` et `s1-eth3`	Filtres d'affichage `arp`, `icmp`, `tcp.port == 8080`

TABLE 3 – Répartition des commandes entre les fenêtres

2 Démarrage de l'environnement

2.1 Terminal 1 : création de la topologie

Listing 1 – Nettoyage puis création du réseau

```
1 sudo mn -c
```

```
sudo mn --topo single,3 --switch ovsk,protocols=OpenFlow13 --
    controller none
```



FIGURE 2 – Nettoyage initial avec `sudo mn -c`



FIGURE 3 – Création de la topologie `single,3` sans contrôleur

Listing 2 – Vérification de la topologie

```
mininet> net
mininet> nodes
mininet> h1 ip -br addr
mininet> h3 ip -br addr
```

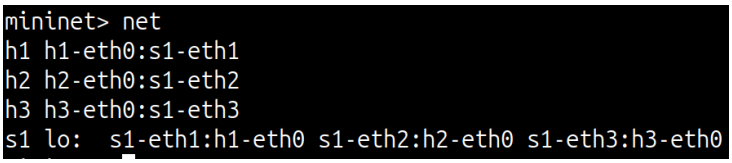


FIGURE 4 – Sortie de `net`

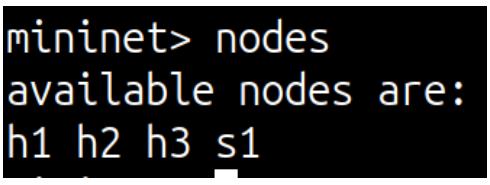


FIGURE 5 – Sortie de `nodes`

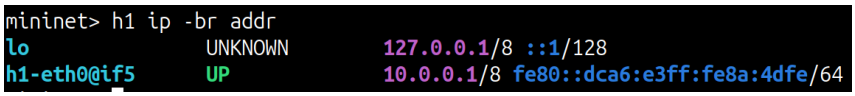


FIGURE 6 – Sortie de `h1 ip -br addr`

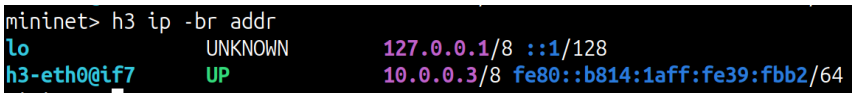


FIGURE 7 – Sortie de `h3 ip -br addr`

Observation. `net` montre `s1` relié aux trois hôtes : `s1-eth1:h1-eth0`, `s1-eth2:h2-eth0` et `s1-eth3:h3-eth0`. Les commandes `ip -br addr` confirment les adresses 10.0.0.1 et 10.0.0.3. L'invite `mininet>` reste affichée : la topologie est active.

2.2 Terminal 2 : paramètres OVS

Listing 3 – Mode de panne, protocole et ports

```
sudo ovs-vsctl set-fail-mode s1 secure
sudo ovs-vsctl set bridge s1 protocols=OpenFlow13
sudo ovs-vsctl get-fail-mode s1
sudo ovs-vsctl get bridge s1 protocols
sudo ovs-ofctl -O OpenFlow13 show s1
```



FIGURE 8 – `set-fail-mode s1 secure`



FIGURE 9 – `set bridge s1 protocols=OpenFlow13`



FIGURE 10 – `get-fail-mode s1`



FIGURE 11 – `get bridge s1 protocols`

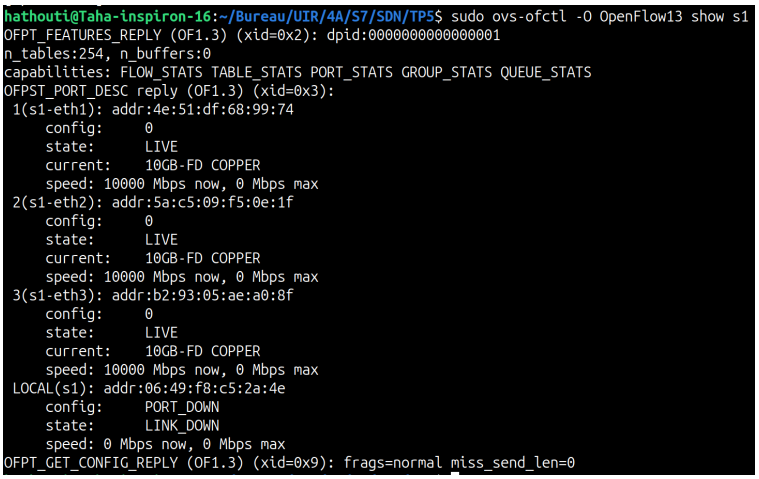


FIGURE 12 – Sortie de `ovs-ofctl -O OpenFlow13 show s1`

Observation. Les réponses sont `secure` et `[OpenFlow13]`. `show s1` répond en OpenFlow 1.3 (`dpid:0000000000000001`, `n_tables:254`) et liste les ports `s1-eth1`, `s1-eth2`, `s1-eth3` à l'état `LIVE` (10 Gbit/s), plus le port interne `LOCAL` (`PORT_DOWN`, sans lien, hors TP). Les numéros 1, 2 et 3 correspondent bien à `h1`, `h2` et `h3`. La ligne `miss_send_len=0` indique qu'aucun paquet n'est envoyé à un contrôleur en l'absence de règle.

2.3 Wireshark

Wireshark est lancé depuis le menu Applications, sans `sudo` (seul `dumpcap` dispose des privilèges de capture). Dans *Capture* → *Options*, seules `s1-eth1` et `s1-eth3` sont cochées, le filtre de capture reste vide et les filtres sont saisis dans la barre d'affichage. Avant chaque test, *Capture* → *Redémarrer* vide les paquets précédents.

3 Expérience 1 — Switch en mode `secure` avec table vide

Objectif : distinguer « paquet reçu sur un port » et « paquet autorisé à ressortir ».

Listing 4 – Terminal 2 : table vide

```
sudo ovs-ofctl -O OpenFlow13 del-flows s1
sudo ovs-ofctl -O OpenFlow13 dump-flows s1
```



FIGURE 13 – Suppression des anciennes règles



FIGURE 14 – Vérification : table de flux vide

Listing 5 – Terminal 1 : trafic (filtre Wireshark : `arp`)

```
mininet> h1 ip neigh flush all
mininet> h3 ip neigh flush all
mininet> h1 ping -c 2 10.0.0.3
```

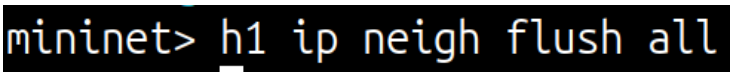


FIGURE 15 – `h1 ip neigh flush all`

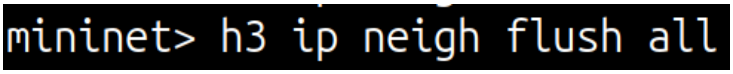


FIGURE 16 – `h3 ip neigh flush all`

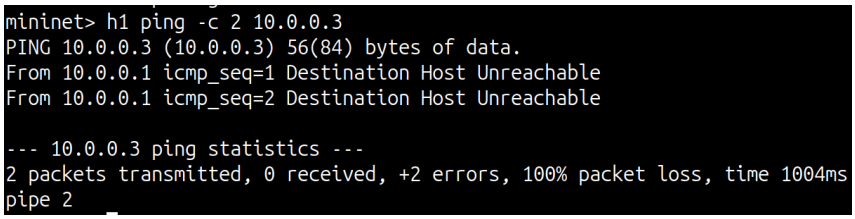


FIGURE 17 – Sortie de `h1 ping -c 2 10.0.0.3`

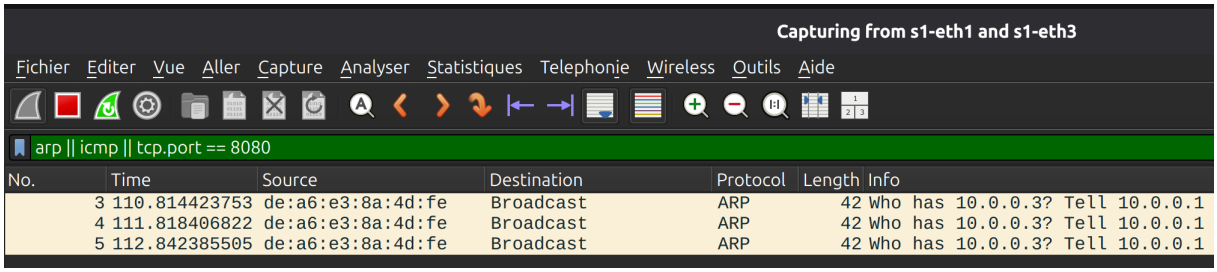


FIGURE 18 – Capture Wireshark (filtre `arp`) sur `s1-eth1` et `s1-eth3`

Observation. La table est vide. Le ping échoue : 2 émis, 0 reçu, `+2 errors`, 100 % de perte, avec `From 10.0.0.1 icmp_seq=1 Destination Host Unreachable`. Dans Wireshark, l'ARP Request de `h1` apparaît sur `s1-eth1` mais pas sur `s1-eth3`.

Question : si l'on voit ARP sur `s1-eth1` mais pas sur `s1-eth3`, est-ce une panne du câble virtuel ?

Non. Le lien fonctionne puisque la trame arrive bien sur le switch (elle est capturée sur `s1-eth1`) et que les ports sont `LIVE`. C'est `s1` qui ne la retransmet pas : en mode `secure` et sans règle, aucune action ne lui dit où l'envoyer. `h1` ne résout pas l'adresse MAC de `h3`, n'émet aucun ICMP et son propre noyau déclare l'échec. Un ping en échec n'est donc pas forcément un problème physique : la cause peut être une politique de transfert.

4 Expérience 2 — Retrouver le comportement d'un switch classique

Objectif : comprendre l'action `NORMAL` et constater qu'une seule règle change la politique de transfert.

Listing 6 – Terminal 2 : règle `NORMAL`

```
sudo ovs-ofctl -O OpenFlow13 add-flow s1 "priority=0,actions=NORMAL
    "
sudo ovs-ofctl -O OpenFlow13 dump-flows s1
```

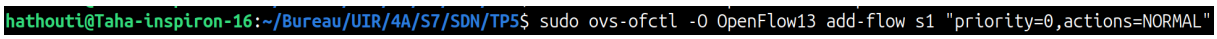


FIGURE 19 – Ajout de `priority=0,actions=NORMAL`

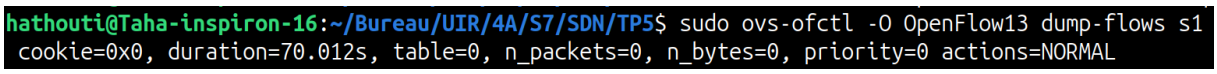


FIGURE 20 – Table de flux après l'ajout

Listing 7 – Terminal 1 : trafic (filtre Wireshark : `arp || icmp`)

```
mininet> h1 ip neigh flush all
mininet> h3 ip neigh flush all
mininet> h1 ping -c 2 10.0.0.3
mininet> pingall
```

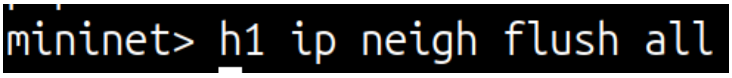


FIGURE 21 – `h1 ip neigh flush all`

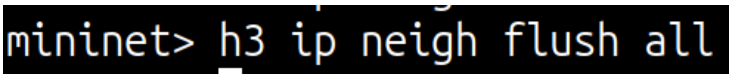


FIGURE 22 – `h3 ip neigh flush all`

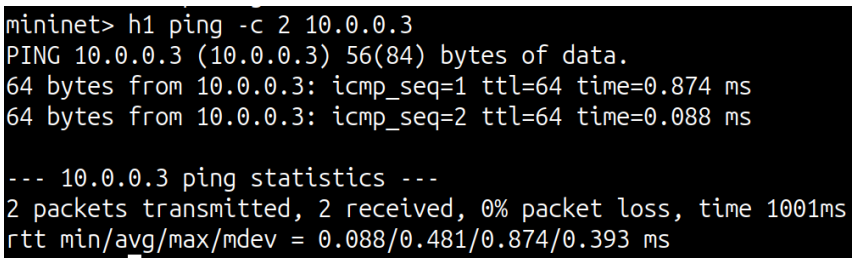


FIGURE 23 – Sortie de `h1 ping -c 2 10.0.0.3`

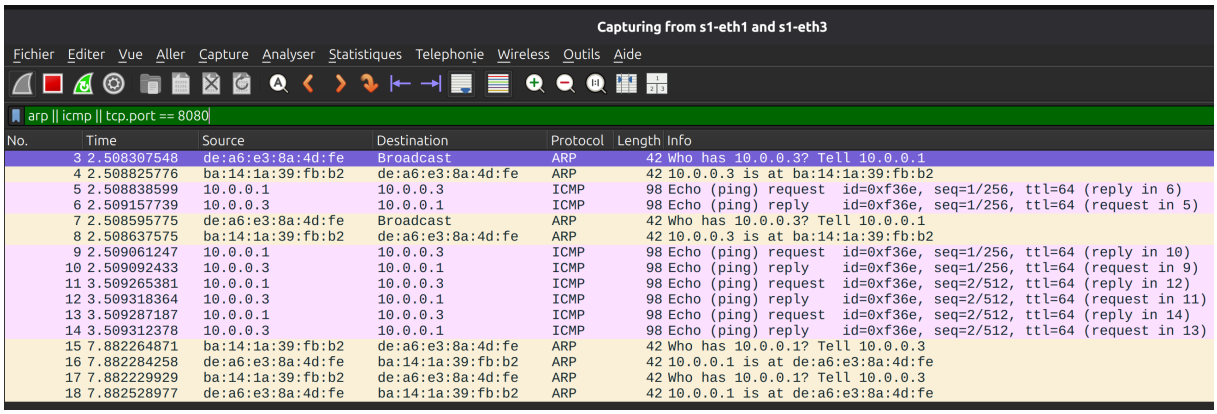


FIGURE 24 – Capture Wireshark du ping `h1` → `h3`

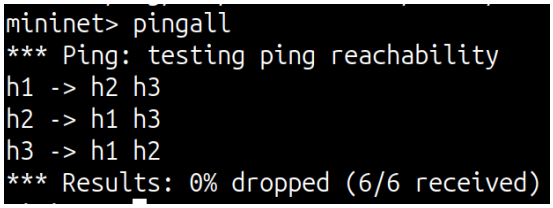


FIGURE 25 – Sortie de `pingall`

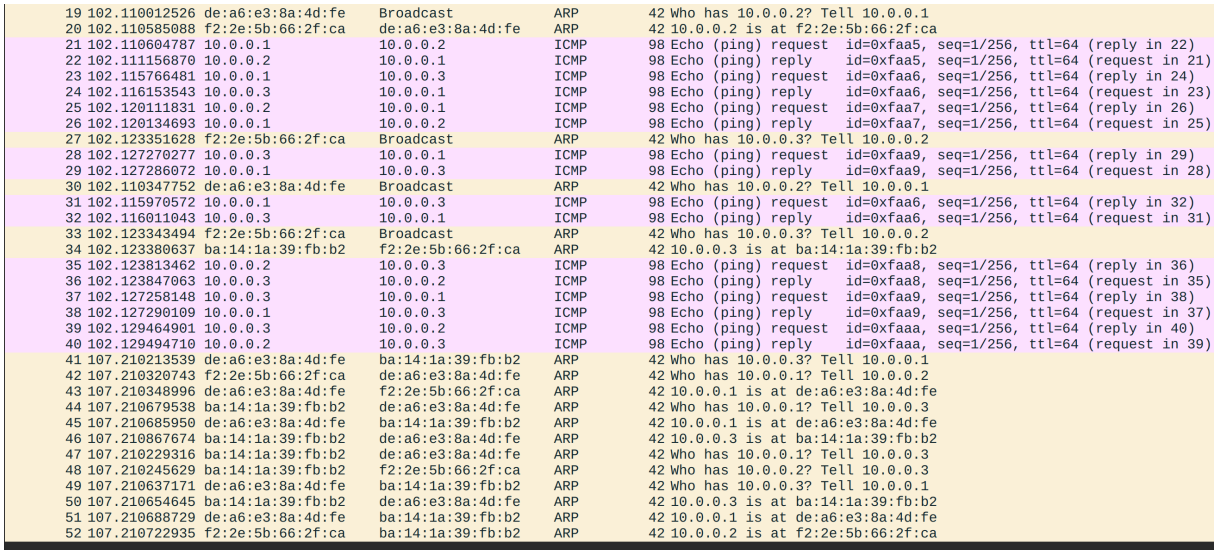


FIGURE 26 – Capture Wireshark pendant `pingall`

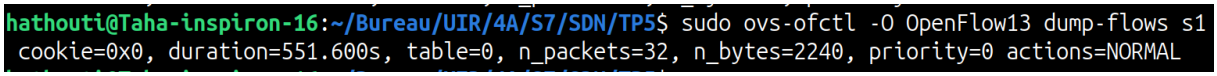


FIGURE 27 – `dump-flows` après les tests : compteur de la règle `NORMAL`

Observation. Le ping `h1` → `h3` réussit : 2/2 reçus, 0 % de perte, RTT min/moyen/max = 0,088/0,481/0,874 ms. `pingall` donne `0% dropped (6/6 received)`. Le compteur de la règle `NORMAL` atteint `n_packets=32`, `n_bytes=2240`.

Question : en quoi `NORMAL` diffère-t-il de `output:N` ?

`NORMAL` déclenche le pipeline de commutation classique d'OVS : apprentissage des adresses MAC, transfert vers le bon port, diffusion des trames inconnues. Le port de sortie est choisi trame par trame, alors que `output:N` impose un port fixe.

Listing 8 – Suppression de la règle `NORMAL` avant les règles explicites

```
sudo ovs-ofctl -O OpenFlow13 del-flows s1
```



FIGURE 28 – Suppression de la règle `NORMAL`



FIGURE 29 – Vérification : table vide

5 Expérience 3 — ARP puis forwarding IPv4 manuel

5.1 Autoriser seulement ARP

Objectif : montrer que la résolution MAC peut fonctionner alors que les paquets IP restent abandonnés.

Listing 9 – Règle ARP

```
sudo ovs-ofctl -O OpenFlow13 add-flow s1 "priority=100,arp,actions=
   FLOOD"
sudo ovs-ofctl -O OpenFlow13 dump-flows s1
```

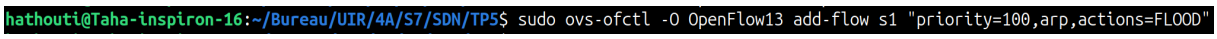


FIGURE 30 – Ajout de la règle `arp,actions=FLOOD`

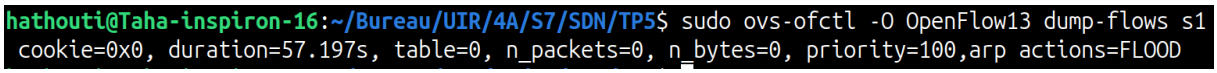


FIGURE 31 – Table de flux après l'ajout

Listing 10 – Terminal 1 : trafic (filtre Wireshark : `arp || icmp`)

```
mininet> h1 ip neigh flush all
mininet> h3 ip neigh flush all
mininet> h1 ping -c 2 10.0.0.3
```

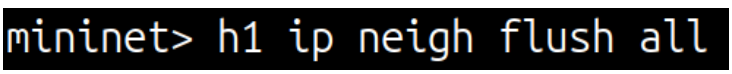


FIGURE 32 – `h1 ip neigh flush all`

```
mininet> h3 ip neigh flush all
```

FIGURE 33 – `h3 ip neigh flush all`

```
mininet> h1 ping -c 2 10.0.0.3
PING 10.0.0.3 (10.0.0.3) 56(84) bytes of data.

--- 10.0.0.3 ping statistics ---
2 packets transmitted, 0 received, 100% packet loss, time 1022ms
```

FIGURE 34 – Sortie de `h1 ping -c 2 10.0.0.3`

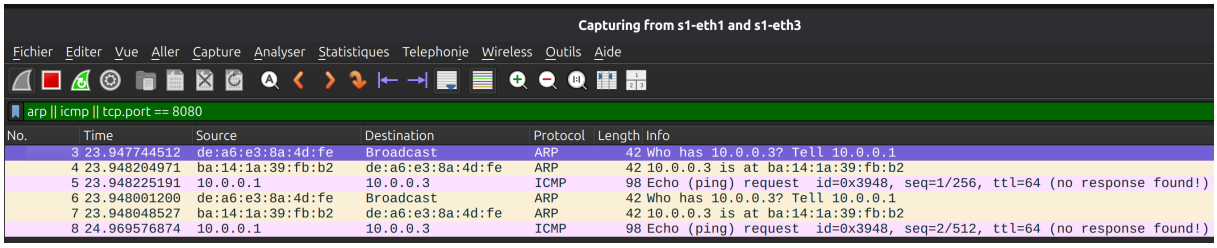


FIGURE 35 – Capture Wireshark : ARP sur les deux ports, ICMP à l'entrée seulement

Observation. Le ping échoue : 2 émis, 0 reçu, 100 % de perte (1022 ms), cette fois **sans** `Destination Host Unreachable`. L'ARP Request/Reply est visible sur les deux ports, mais les Echo Request n'apparaissent que sur `s1-eth1`.

Interprétation. `h1` apprend l'adresse MAC de `h3` et envoie réellement ses paquets ICMP, mais ceux-ci ne correspondent à aucune règle et sont abandonnés par le switch : **l'ARP ne suffit pas à faire réussir un ping**.

5.2 Autoriser IPv4 selon l'adresse de destination

Objectif : construire un forwarding explicite vers chaque hôte, sans `NORMAL`.

Listing 11 – Vérification des ports puis trois règles IPv4

```
1 sudo ovs-ofctl -O OpenFlow13 show s1
2 sudo ovs-ofctl -O OpenFlow13 add-flow s1 "priority=200,ip,nw_dst
     =10.0.0.1,actions=output:1"
3 sudo ovs-ofctl -O OpenFlow13 add-flow s1 "priority=200,ip,nw_dst
     =10.0.0.2,actions=output:2"
4 sudo ovs-ofctl -O OpenFlow13 add-flow s1 "priority=200,ip,nw_dst
     =10.0.0.3,actions=output:3"
```

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP5$ sudo ovs-ofctl -O OpenFlow13 show s1
[sudo: authenticate] Mot de passe :
OFPT_FEATURES_REPLY (OF1.3) (xid=0x2): dpid:0000000000000001
n_tables:254, n_buffers:0
capabilities: FLOW_STATS TABLE_STATS PORT_STATS GROUP_STATS QUEUE_STATS
OFPST_PORT_DESC reply (OF1.3) (xid=0x3):
 1(s1-eth1): addr:4e:51:df:68:99:74
     config:     0
     state:      LIVE
     current:    10GB-FD COPPER
     speed: 10000 Mbps now, 0 Mbps max
 2(s1-eth2): addr:5a:c5:09:f5:0e:1f
     config:     0
     state:      LIVE
     current:    10GB-FD COPPER
     speed: 10000 Mbps now, 0 Mbps max
 3(s1-eth3): addr:b2:93:05:ae:a0:8f
     config:     0
     state:      LIVE
     current:    10GB-FD COPPER
     speed: 10000 Mbps now, 0 Mbps max
 LOCAL(s1): addr:06:49:f8:c5:2a:4e
     config:     PORT_DOWN
     state:      LINK_DOWN
     speed: 0 Mbps now, 0 Mbps max
OFPT_GET_CONFIG_REPLY (OF1.3) (xid=0x9): frags=normal miss_send_len=0
```

FIGURE 36 – Vérification des ports avec `show s1`

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP5$ sudo ovs-ofctl -O OpenFlow13 add-flow s1 "priority=200,ip,nw_dst=10.0.0.1,actions=output:1"
```

FIGURE 37 – Règle `nw_dst=10.0.0.1` → `output:1`

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP5$ sudo ovs-ofctl -O OpenFlow13 add-flow s1 "priority=200,ip,nw_dst=10.0.0.2,actions=output:2"
```

FIGURE 38 – Règle `nw_dst=10.0.0.2` → `output:2`

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP5$ sudo ovs-ofctl -O OpenFlow13 add-flow s1 "priority=200,ip,nw_dst=10.0.0.3,actions=output:3"
```

FIGURE 39 – Règle `nw_dst=10.0.0.3` → `output:3`

Listing 12 – Terminal 1 : trafic (filtre Wireshark : `icmp`)

```
1 mininet> h1 ping -c 2 10.0.0.3
2 mininet> pingall
```

```
mininet> h1 ping -c 2 10.0.0.3
PING 10.0.0.3 (10.0.0.3) 56(84) bytes of data.
64 bytes from 10.0.0.3: icmp_seq=1 ttl=64 time=0.617 ms
64 bytes from 10.0.0.3: icmp_seq=2 ttl=64 time=0.109 ms

--- 10.0.0.3 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1057ms
rtt min/avg/max/mdev = 0.109/0.363/0.617/0.254 ms
```

FIGURE 40 – Sortie de `h1 ping -c 2 10.0.0.3`

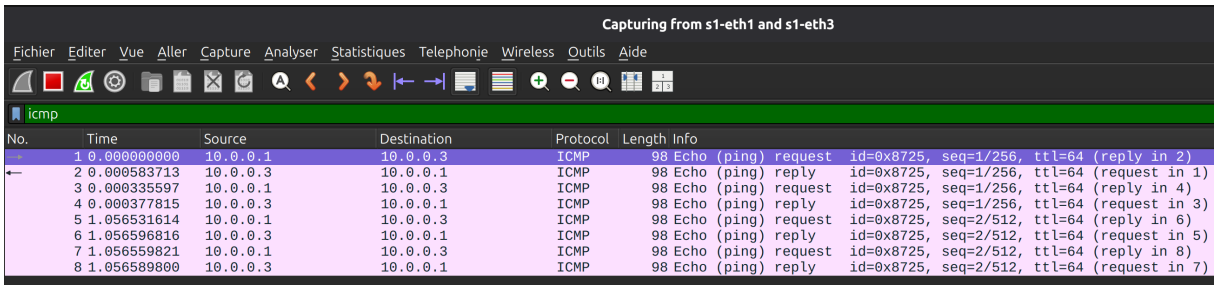


FIGURE 41 – Capture Wireshark (filtre `icmp`) du ping `h1` → `h3`

```
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 h3
h2 -> h1 h3
h3 -> h1 h2
*** Results: 0% dropped (6/6 received)
```

FIGURE 42 – Sortie de `pingall`

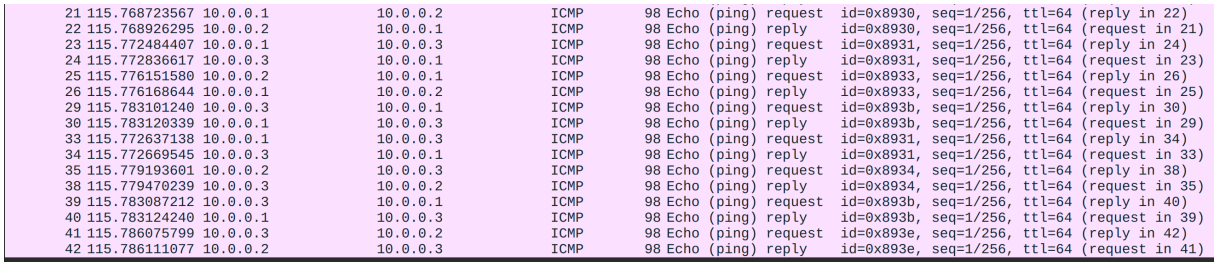


FIGURE 43 – Capture Wireshark pendant `pingall`

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP5$ sudo ovs-ofctl -O OpenFlow13 dump-flows s1
 cookie=0x0, duration=465.522s, table=0, n_packets=6, n_bytes=588, priority=200,ip,nw_dst=10.0.0.1 actions=output:"s1-eth1"
 cookie=0x0, duration=413.092s, table=0, n_packets=4, n_bytes=392, priority=200,ip,nw_dst=10.0.0.2 actions=output:"s1-eth2"
 cookie=0x0, duration=377.756s, table=0, n_packets=6, n_bytes=588, priority=200,ip,nw_dst=10.0.0.3 actions=output:"s1-eth3"
 cookie=0x0, duration=2856.943s, table=0, n_packets=18, n_bytes=756, priority=100,arp actions=FLOOD
```

FIGURE 44 – Table de flux : 1 règle ARP et 3 règles IPv4

Observation. Le ping `h1` → `h3` réussit : 2/2 reçus, RTT min/moyen/max = 0,109/0,363/0,617 ms, et `pingall` donne 6/6. Compteurs relevés :

Priorité	Règle	n_packets	n_bytes
200	`ip,nw_dst=10.0.0.1` → `s1-eth1`	6	588
200	`ip,nw_dst=10.0.0.2` → `s1-eth2`	4	392
200	`ip,nw_dst=10.0.0.3` → `s1-eth3`	6	588
100	`arp` → `FLOOD`	18	756

TABLE 4 – Compteurs de `dump-flows` (expérience 3)

Les compteurs sont cumulatifs depuis l'installation des règles : ils ne correspondent pas au seul dernier ping.

Champ	Exemple	Interprétation
`table`	`table=0`	Table OpenFlow contenant la règle
`priority`	200	Plus élevé : sélectionné parmi les règles qui correspondent
match	`ip,nw_dst=10.0.0.3`	Paquet IPv4 destiné à `h3`
`actions`	`output:"s1-eth3"`	Sortie vers le port 3
`n_packets`, `n_bytes`	6, 588	Paquets et octets cumulés

TABLE 5 – Champs d'une règle affichée par `dump-flows`

6 Expérience 4 — ACL ICMP et priorités

6.1 Bloquer tout ICMP de h1 vers h3

Objectif : montrer qu'une règle spécifique (priorité 300) l'emporte sur la règle générale IPv4 (priorité 200).

Listing 13 – Règle de blocage ICMP `h1` → `h3`

```
sudo ovs-ofctl -O OpenFlow13 add-flow s1 "priority=300,icmp,nw_src
   =10.0.0.1,nw_dst=10.0.0.3,actions=drop"
```

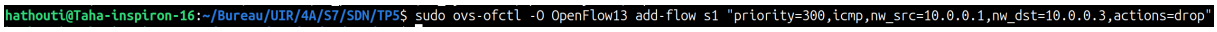


FIGURE 45 – Ajout de la règle `priority=300` / `drop`

Listing 14 – Test (filtre Wireshark : `icmp && ip.src == 10.0.0.1 && ip.dst == 10.0.0.3`)

```
mininet> h1 ping -c 3 10.0.0.3
```

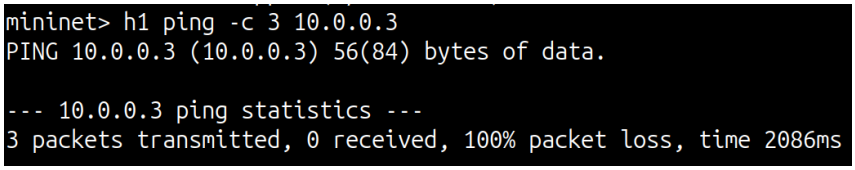


FIGURE 46 – Sortie de `h1 ping -c 3 10.0.0.3`

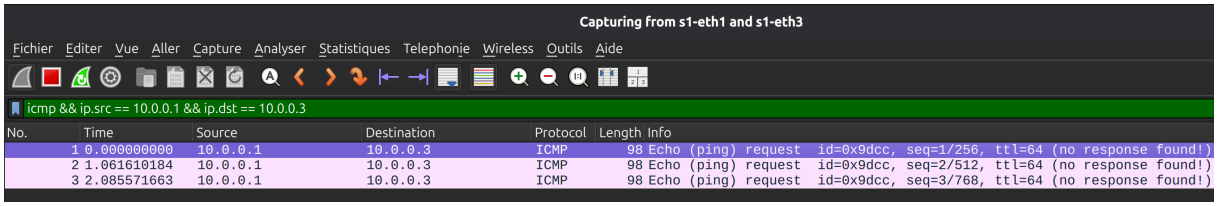


FIGURE 47 – Capture Wireshark : Echo Request sur `s1-eth1` uniquement

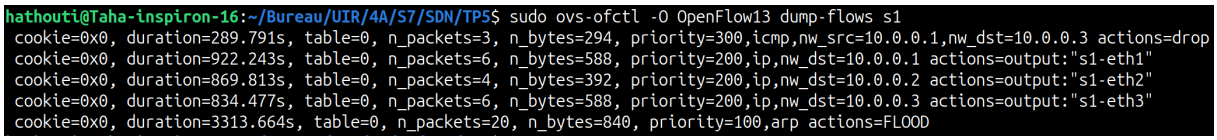


FIGURE 48 – Table de flux avec la règle `priority=300`

Observation. Le ping échoue : 3 émis, 0 reçu, 100 % de perte (2086 ms), sans `Destination Host Unreachable` car ARP reste autorisé. La règle `priority=300` affiche `n_packets=3`, `n_bytes=294`.

Pourquoi la priorité 300 l'emporte-t-elle ? Un paquet ICMP `h1` → `h3` correspond à la fois à la règle IPv4 (priorité 200) et à la règle de blocage (priorité 300) ; c'est la priorité la plus élevée qui s'applique.

6.2 Supprimer précisément cette ACL

Listing 15 – Suppression stricte

```
sudo ovs-ofctl -O OpenFlow13 --strict del-flows s1 "priority=300,
   icmp,nw_src=10.0.0.1,nw_dst=10.0.0.3"
sudo ovs-ofctl -O OpenFlow13 dump-flows s1
```

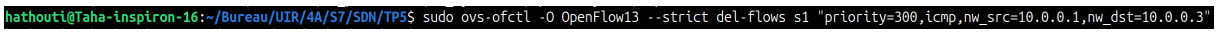


FIGURE 49 – Suppression de la règle `priority=300`

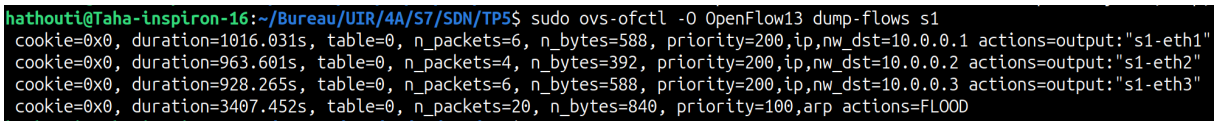


FIGURE 50 – Vérification de la table après suppression

Observation. `-strict` exige la même priorité et les mêmes champs, sans écrire `actions` dans le filtre de suppression : seule cette règle disparaît.

6.3 Bloquer uniquement l'Echo Request (ICMP type 8)

Listing 16 – Règle précise avec `icmp_type=8`

```
sudo ovs-ofctl -O OpenFlow13 add-flow s1 "priority=300,icmp,nw_src
   =10.0.0.1,nw_dst=10.0.0.3,icmp_type=8,actions=drop"
```

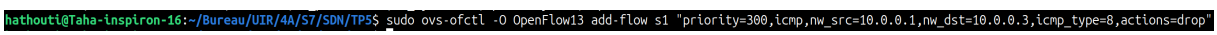


FIGURE 51 – Ajout de la règle bloquant l'Echo Request `h1` → `h3`

Listing 17 – Tests (filtre Wireshark : `icmp`)

```
mininet> h1 ping -c 3 10.0.0.3
mininet> h3 ping -c 3 10.0.0.1
```

```
mininet> h1 ping -c 3 10.0.0.3
PING 10.0.0.3 (10.0.0.3) 56(84) bytes of data.

--- 10.0.0.3 ping statistics ---
3 packets transmitted, 0 received, 100% packet loss, time 2069ms
```

FIGURE 52 – Sortie de `h1 ping -c 3 10.0.0.3`

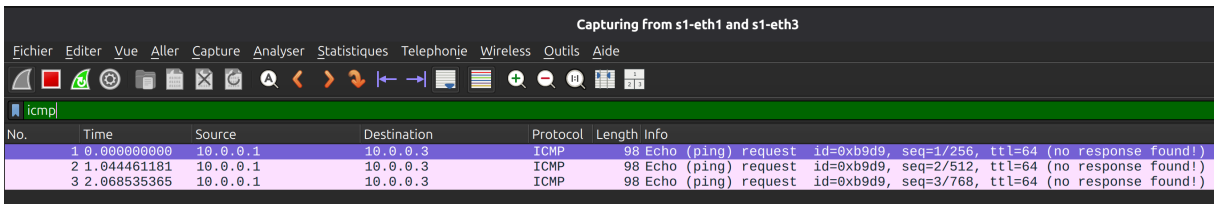


FIGURE 53 – Capture Wireshark du ping `h1` → `h3`

```
mininet> h3 ping -c 3 10.0.0.1
PING 10.0.0.1 (10.0.0.1) 56(84) bytes of data.
64 bytes from 10.0.0.1: icmp_seq=1 ttl=64 time=0.671 ms
64 bytes from 10.0.0.1: icmp_seq=2 ttl=64 time=0.056 ms
64 bytes from 10.0.0.1: icmp_seq=3 ttl=64 time=0.074 ms

--- 10.0.0.1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2075ms
rtt min/avg/max/mdev = 0.056/0.267/0.671/0.285 ms
```

FIGURE 54 – Sortie de `h3 ping -c 3 10.0.0.1`

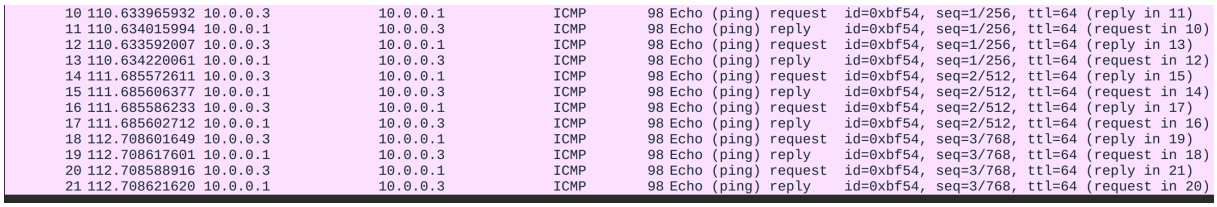


FIGURE 55 – Capture Wireshark du ping `h3` → `h1`

Observation.

— `h1` → `h3` : 3 émis, 0 reçu, 100 % de perte (2069 ms) : l'Echo Request est bloqué à l'entrée ;

— `h3` → `h1` : 3/3 reçus, RTT min/moyen/max = 0,056/0,267/0,671 ms.

Apport de `icmp_type=8`. Le match est affiné : on bloque seulement les demandes d'écho d'un sens, au lieu de tout le protocole ICMP.

Listing 18 – Retour au fonctionnement normal avant TCP

```
sudo ovs-ofctl -O OpenFlow13 --strict del-flows s1 "priority=300,
   icmp,nw_src=10.0.0.1,nw_dst=10.0.0.3,icmp_type=8"
```

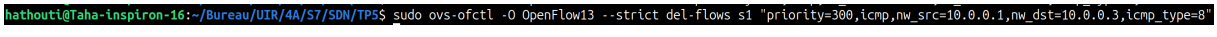


FIGURE 56 – Suppression de la règle `icmp_type=8`

7 Expérience 5 — ACL TCP sur le service HTTP 8080

Objectif : filtrer un service applicatif précis sans empêcher le reste du réseau de fonctionner. Le serveur HTTP s'exécute dans `h3`.

7.1 Démarrer le serveur et vérifier le trafic autorisé

Listing 19 – Terminal 1 : serveur HTTP puis requête (filtre Wireshark : `tcp.port == 8080`)

```
mininet> h3 python3 -m http.server 8080 > /tmp/http_tp.log 2>&1 &
mininet> h1 curl -I --max-time 5 http://10.0.0.3:8080
```

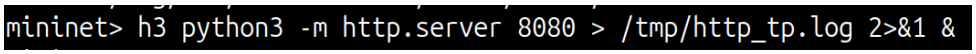


FIGURE 57 – Démarrage du serveur HTTP dans `h3`

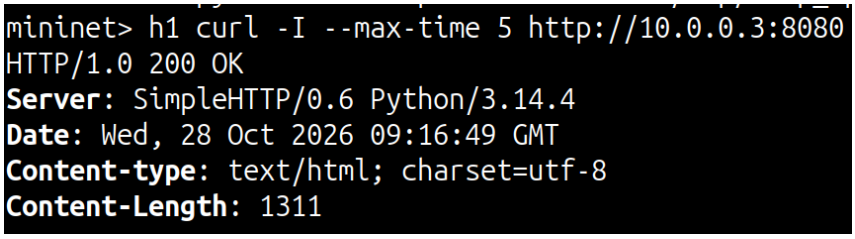


FIGURE 58 – Sortie de `h1 curl -I http://10.0.0.3:8080`

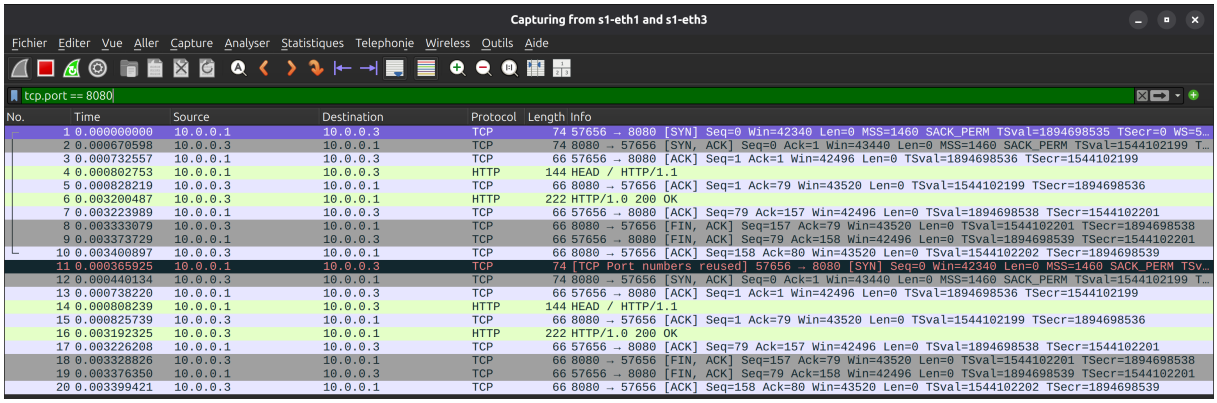


FIGURE 59 – Capture Wireshark (filtre `tcp.port == 8080`)

Observation. `curl` reçoit `HTTP/1.0 200 OK`. Dans Wireshark, SYN, SYN/ACK et ACK traversent les deux interfaces, suivis de l'échange HTTP.

7.2 Bloquer TCP vers 10.0.0.3 :8080

Listing 20 – Règle de blocage du service

```
sudo ovs-ofctl -O OpenFlow13 add-flow s1 "priority=350,tcp,nw_src
   =10.0.0.1,nw_dst=10.0.0.3,tcp_dst=8080,actions=drop"
```

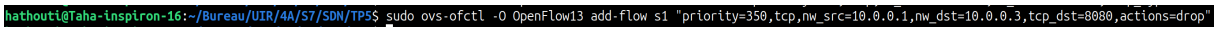


FIGURE 60 – Ajout de la règle `priority=350` / `drop`

Listing 21 – Test et vérification

```
mininet> h1 curl -I --max-time 3 http://10.0.0.3:8080
sudo ovs-ofctl -O OpenFlow13 dump-flows s1
```

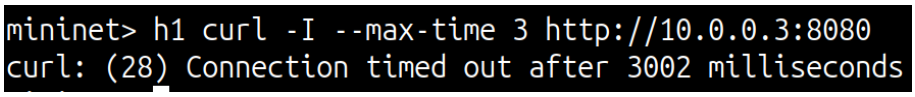


FIGURE 61 – Sortie de `curl` avec la règle active

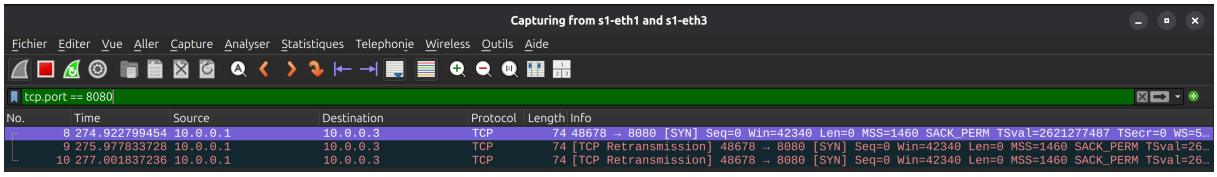


FIGURE 62 – Capture Wireshark : SYN sur `s1-eth1` uniquement

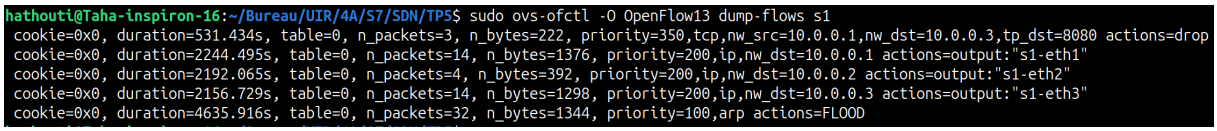


FIGURE 63 – Table de flux avec la règle `priority=350`

Observation. `curl: (28) Connection timed out after 3002 milliseconds`. Les SYN de `h1` sont présents sur `s1-eth1` et absents de `s1-eth3`. La règle `priority=350` affiche `n_packets=3`, `n_bytes=222` : les trois SYN (premier et retransmissions) ont été supprimés.

7.3 Supprimer l'ACL et restaurer HTTP

Listing 22 – Suppression ciblée puis nouvelle requête

```
sudo ovs-ofctl -O OpenFlow13 --strict del-flows s1 "priority=350,
   tcp,nw_src=10.0.0.1,nw_dst=10.0.0.3,tcp_dst=8080"
mininet> h1 curl -I --max-time 5 http://10.0.0.3:8080
```

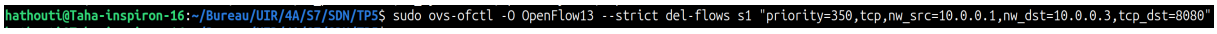


FIGURE 64 – Suppression de la règle `priority=350`

```
mininet> h1 curl -I --max-time 5 http://10.0.0.3:8080
HTTP/1.0 200 OK
Server: SimpleHTTP/0.6 Python/3.14.4
Date: Wed, 28 Oct 2026 09:30:16 GMT
Content-type: text/html; charset=utf-8
Content-Length: 1631
```

FIGURE 65 – Sortie de `curl` après la suppression

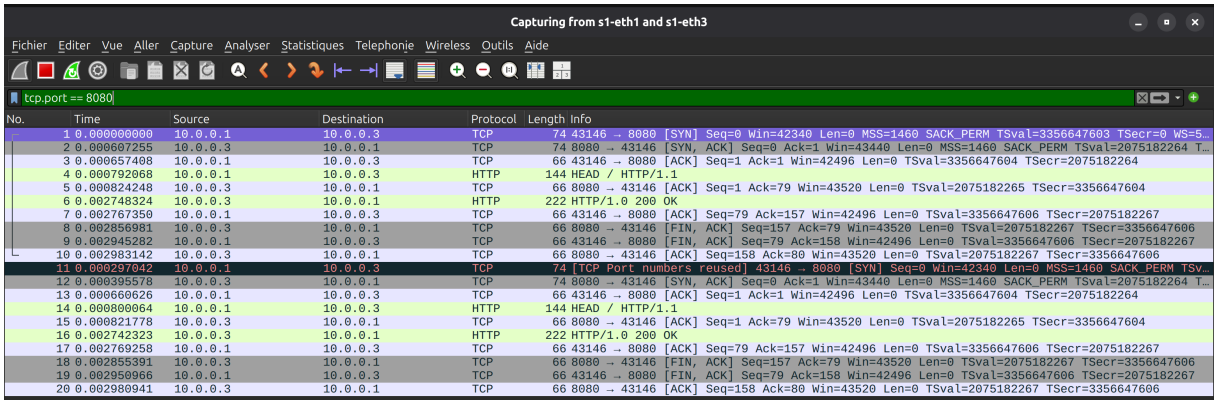


FIGURE 66 – Capture Wireshark après la suppression

Observation. Le serveur répond à nouveau `HTTP/1.0 200 OK` ; SYN et réponse HTTP réapparaissent sur les deux interfaces.

8 Synthèse Wireshark — comparer les états A à D

État	**Règles installées**	**Wireshark** (`s1-eth1` / `s1-eth3`)
A — table vide	Aucune	ARP Request seulement sur `s1-eth1`
B — ARP seul	`priority=100,arp,FLOOD`	ARP sur les deux ; ICMP uniquement à l'entrée
C — forwarding	ARP + 3 flux IPv4 `output:1/2/3`	Echo Request et Reply des deux côtés
D — DROP	Forwarding + ACL ICMP type 8	Echo Request sur l'entrée, pas sur la sortie

TABLE 6 – États observés dans Wireshark

Filtre d'affichage	Utilisation
`arp`	Résolution d'adresses MAC
`icmp`	Paquets ping (Echo Request / Echo Reply)
`tcp.port == 8080`	Service HTTP de `h3`
`icmp && ip.src == 10.0.0.1 && ip.dst == 10.0.0.3`	ICMP d'un seul sens
`ip.addr == 10.0.0.1 || ip.addr == 10.0.0.3`	Trafic impliquant `h1` ou `h3`

TABLE 7 – Filtres d'affichage à maîtriser

8.1 Observation de `pingall` et des tables 0 et 1

Listing 23 – Trafic et lecture des tables

```
mininet> pingall
sudo ovs-ofctl -O OpenFlow13 dump-flows s1 "table=0"
sudo ovs-ofctl -O OpenFlow13 dump-flows s1 "table=1"
```

```
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 h3
h2 -> h1 h3
h3 -> h1 h2
*** Results: 0% dropped (6/6 received)
```

FIGURE 67 – Sortie de `pingall`

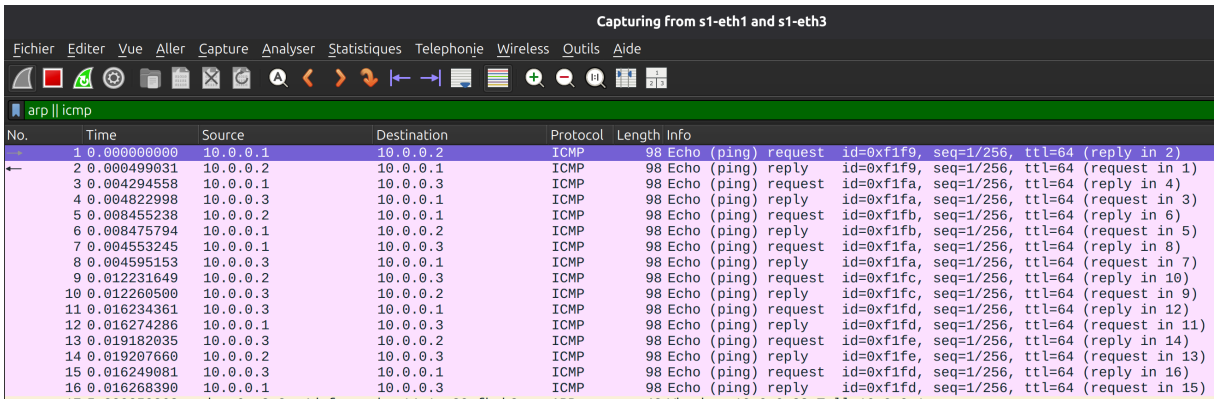


FIGURE 68 – Capture Wireshark pendant `pingall` (filtre `arp || icmp`)

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP5$ sudo ovs-ofctl -O OpenFlow13 dump-flows s1 "table=0"
 cookie=0x0, duration=2731.831s, table=0, n_packets=23, n_bytes=2262, priority=200,ip,nw_dst=10.0.0.1 actions=output:"s1-eth1"
 cookie=0x0, duration=2679.401s, table=0, n_packets=8, n_bytes=784, priority=200,ip,nw_dst=10.0.0.2 actions=output:"s1-eth2"
 cookie=0x0, duration=2644.065s, table=0, n_packets=23, n_bytes=2106, priority=200,ip,nw_dst=10.0.0.3 actions=output:"s1-eth3"
 cookie=0x0, duration=5123.252s, table=0, n_packets=48, n_bytes=2016, priority=100,arp actions=FLOOD
```

FIGURE 69 – `dump-flows s1 "table=0"`

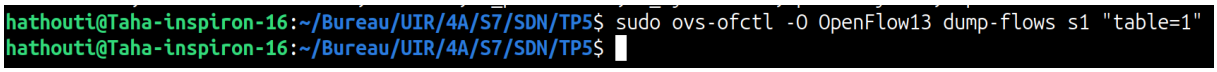


FIGURE 70 – `dump-flows s1 "table=1"`

Observation. `pingall` donne `0% dropped (6/6 received)`. La table 0 contient les trois règles IPv4 de priorité 200 et la règle ARP `FLOOD`. La table 1 est **vide**.

Remarque. L'énoncé de cette partie annonçait une séparation de la classification et du forwarding en deux tables, mais ne contient pas l'étape de création et de remplissage de la table 1 (oubli du sujet, confirmé à l'enseignant qui a indiqué que ce n'était pas nécessaire). Tout le transfert reste donc réalisé par la table 0.

9 Clôture du TP

Ordre obligatoire : arrêter et enregistrer la capture Wireshark **avant** de nettoyer Mininet, sinon les interfaces de capture disparaissent.

Listing 24 – Fermeture de Mininet puis nettoyage

```
1 mininet> exit
2 sudo mn -c
```

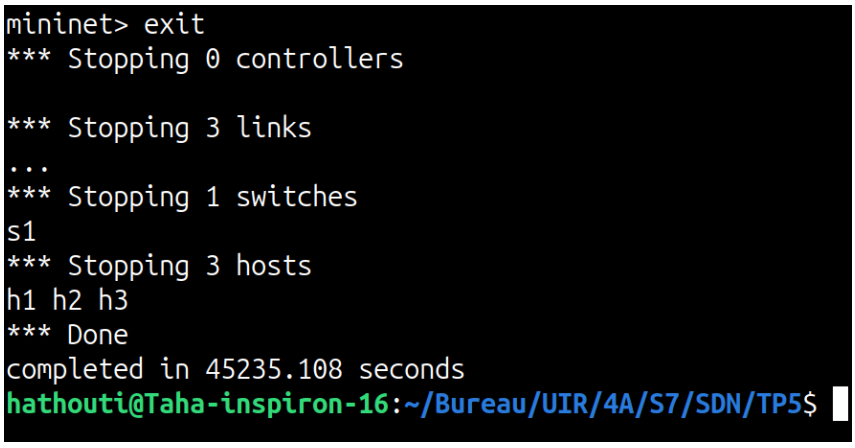


FIGURE 71 – Fermeture de Mininet avec `exit`

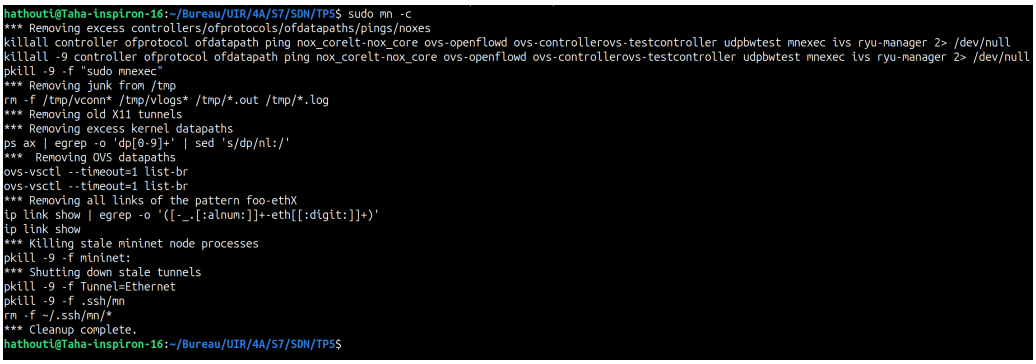


FIGURE 72 – Nettoyage final avec `sudo mn -c`

Observation. `exit` arrête les contrôleurs, les 3 liens, le switch `s1` et les 3 hôtes, puis Mininet affiche `Done`. `sudo mn -c` nettoie les interfaces virtuelles.

10 Questions de validation

1. **Pourquoi `s1` ne transmet-il pas à l'expérience 1 alors que ses interfaces sont UP ?** En mode `secure`, sans règle, aucun paquet n'est transféré : un lien actif ne suffit pas, il faut une règle qui indique l'action.
2. **Rôle de `ovs-vsctl` et `ovs-ofctl` ?** `ovs-vsctl` configure OVS (bridges, ports, `fail-mode`, protocoles) ; `ovs-ofctl` gère OpenFlow et donc les flux.
3. **Rôle de `-O OpenFlow13` ?** Il impose la version 1.3 du protocole, cohérente avec `protocols=OpenFlow13`.
4. **Pourquoi vérifier les ports avant `output:3` ?** Les numéros dépendent de l'ordre de création des liens ; `show s1` fait foi.
5. **Pourquoi ARP seul ne suffit-il pas ?** Il ne fournit que la résolution MAC ; les paquets ICMP n'ont encore aucune règle et sont abandonnés.
6. **Que démontre 300 contre 200 ?** Quand plusieurs règles correspondent, celle de plus haute priorité s'applique.
7. **Un Echo Request sur `s1-eth1` mais pas sur `s1-eth3` prouve quoi ?** Que le paquet est arrivé au switch mais n'a pas été retransmis.
8. **Apport de `icmp_type=8` ?** Il ne bloque que les Echo Request et laisse passer les Echo Reply et le ping dans l'autre sens.
9. **Quel paquet TCP observe-t-on avec le `DROP` actif ?** Uniquement des SYN de `h1` (premier et retransmissions), sans SYN/ACK.
10. **Pourquoi `n_packets` ne correspond-il pas à `-c 3` ?** Les compteurs sont cumulatifs depuis l'installation de la règle.

11 Synthèse

Exp.	Politique	Règles	Résultat observé
1	Table vide	Aucune	Ping en échec, `Host Unreachable`
2	Switch classique	`priority=0,NORMAL`	`pingall` 6/6
3A	ARP seul	`priority=100,arp,FLOOD`	ARP OK, ping 100 % de perte
3B	Forwarding IPv4	3 règles `priority=200`	`pingall` 6/6
4	ACL ICMP	`priority=300`, `icmp_type=8`	`h1`→`h3` bloqué, `h3`→`h1` OK
5	ACL TCP 8080	`priority=350`	`curl` timeout, puis `200 OK` après suppression

Table 8 – Récapitulatif des expériences

Réponse à la question de départ. Non, un ping qui échoue n'est pas forcément un problème physique : les liens sont `LIVE` et les paquets arrivent au switch, mais c'est la table de flux qui décide s'ils ressortent. Wireshark sur l'entrée et la sortie, associé aux compteurs de `dump-flows`, permet de localiser la cause.

Conclusion. À la fin du TP, je sais installer, modifier et supprimer des flux OpenFlow à la main dans Open vSwitch, expliquer pourquoi une règle l'emporte sur une autre grâce à la priorité, filtrer ARP, ICMP (par type) et TCP (par port), et vérifier le comportement du switch avec les messages du ping, les compteurs de `dump-flows` et Wireshark.